ASSIGNMENT 2: IMAGE STITCHING

Nahiyan Al Mahmood | 1155169 | nalmahm@lakeheadu.ca

Introduction:

The process of seamlessly combining multiple photographic images with overlapping fields of view to create a panoramic or high-resolution composite image is known as image stitching, a fundamental task in computer vision. This technique finds applications in various domains, including photography, cartography, and virtual reality, where it generates immersive and expansive visuals.

For this assignment, we are tasked with implementing an image stitching solution specifically designed to run on the Xilinx PYNQ board. The board offers an embedded platform for hardware/software co-design, making it perfect for real-time image processing applications. Our objective is to stitch three input images received on the PYNQ board and present the result in a unified display, with the input images arranged at the top and the stitched image positioned at the bottom.

This report provides an overview of the methodology, implementation details, and results of the developed image-stitching solution. The aim is to provide a comprehensive understanding of the chosen algorithm, its adaptation for the PYNQ board, and the overall workflow leading to the successful generation of a stitched image. The solution leverages computer vision techniques to achieve seamless integration of the input images.

Methodology:

Image Stitching Algorithm

The image stitching algorithm implemented in this project is based on the OpenCV library, a powerful tool for computer vision and image processing tasks. The chosen approach involves the use of OpenCV's Stitcher module, which provides a high-level interface for stitching images seamlessly. The algorithm operates on a sequence of input images and attempts to combine them into a single panoramic or high-resolution image.

Resolution Optimization

Before initiating the stitching process, the input images undergo a resolution optimization step. The “get_optimized_resolution” function is responsible for determining an optimized width and height for each image, ensuring that the dimensions adhere to the maximum resolution constraints specified for the Xilinx

PYNQ Z2 board. This function calculates the aspect ratio of the image and adjusts the width and height accordingly to fit within the constraints.

Stitching Process

The core of the image stitching algorithm lies in the "stitch_images" function. This function iterates through the sequence of images, attempting to stitch each pair of consecutive images together. If stitching is successful, it updates the next image in the list with the stitched result. In the case of stitching failures, the function incorporates a retry mechanism, allowing a specified number of retries to address potential issues.

Implementation:

The implementation of the image stitching solution is carried out in a Python script that leverages the capabilities of the OpenCV library. Below is a breakdown of the key components and steps involved in the implementation:

1. Resolution Optimization Function

The "get_optimized_resolution" function is responsible for calculating an optimized resolution for each input image. This function is crucial for ensuring that the image dimensions align with the constraints of the Xilinx PYNQ Z2 board. The calculated resolutions are then used to resize the images before the stitching process.

2. Image Stitching Function

The "stitch_images" function orchestrates the stitching process using OpenCV's Stitcher module. It iterates through the input image sequence, attempting to stitch consecutive pairs of images. In the event of stitching failures, the function implements a retry mechanism to enhance robustness. The final stitched image is obtained as the last image in the processed sequence.

3. Main Execution

The main execution section of the script orchestrates the overall process. It reads a list of image files, processes each image by optimizing its resolution, displays the input images, performs the stitching, and displays the resulting stitched image. The implementation adheres to the specified requirements, ensuring compatibility with the Xilinx PYNQ board.

The code is structured to handle exceptions gracefully, providing informative messages in case of errors during image processing or stitching. The utilization of Matplotlib enables visual inspection of both input images and the final stitched result.

The following sections present a more detailed examination of the implemented functions, results, and analysis, offering deeper insights into the project's inner workings and outcomes.

Importing Required Libraries:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In this section, essential libraries are imported to facilitate image processing and visualization. The ‘cv2’ library, short for OpenCV, serves as a powerful tool for computer vision tasks and image processing. ‘numpy’ is used for numerical operations, offering efficient handling of arrays and matrices. Lastly, ‘matplotlib.pyplot’ provides functions for creating visualizations, particularly useful for displaying images in this context.

Function: get_optimized_resolution

```
def get_optimized_resolution(img_shape, max_resolution=(640, 480)):
    try:
        # Function logic here
    except Exception as e:
        print(f"Error in get_optimized_resolution: {e}")
        return None
```

The ‘get_optimized_resolution’ function is designed to calculate an optimized resolution for an input image. It takes the shape of the image (img_shape) and an optional maximum resolution parameter (max_resolution). The function first checks if the image dimensions exceed the specified maximum resolution. If they do, it calculates a new width and height while maintaining the aspect ratio to fit within the constraints. Exception handling is implemented to catch any errors during execution, and if successful, the function returns the optimized width and height; otherwise, it returns ‘None’.

Function: stitch_images

```
def stitch_images(images, max_retries=3):
    stitcher = cv2.Stitcher_create()
    for i in range(len(images) - 1):
        retries = 0
        while retries < max_retries:
            try:
                # Stitching process logic here
            except Exception as e:
                print(f"Error in stitching (Retry {retries + 1}): {e}")
                retries += 1
    return images[-1]
```

The ‘stitch_images’ function is the core of the ‘image stitching’ process. It utilizes OpenCV's Stitcher module to stitch a sequence of images together. The function iterates through pairs of consecutive images, attempting to stitch them. A retry mechanism is implemented to address stitching failures, with the number of retries controlled by the ‘max_retries’ parameter. The final stitched image is returned, and the function prints informative messages about the stitching process, including any errors that may occur.

Main Execution:

```
if __name__ == "__main__":
    # List of image files to be stitched
    image_files = ['img11.jpg', 'img22.jpg', 'img33.jpg']
    images = []
    try:
        # Image processing loop
        for img_file in image_files:
            # Reading and processing each image
            # Exception handling for image processing
    except Exception as e:
        print(f"Error processing images: {e}")

    if images:
        # Display input images using Matplotlib
        # Stitching images and displaying the result
    else:
        print("No valid images to process.")
```

The main execution section is where the core functionality of the image stitching script is executed. It begins with a check to see if the script is being run as the main program. A list of image files to be stitched is defined (image_files). A loop then processes each image in the list, reading it, optimizing its resolution using the 'get_optimized_resolution' function, and appending it to the 'images' list. Exception handling is in place to catch any errors that may occur during image processing. If valid images are present, the input images are displayed using Matplotlib, and the 'stitch_images' function is called to perform the stitching process.

Visualization (Matplotlib):

```
# Displaying input images
for i, image in enumerate(images):
    plt.subplot(1, len(images), i + 1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title(f"Input Image {i + 1}")
    plt.axis('off')
plt.show()

# Stitching images and displaying the result
result_image = stitch_images(images)

# Displaying the stitched image
if result_image is not None:
```

```
    cv2.imwrite("stitched_image.jpg", result_image)
    plt.imshow(cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB))
    plt.title("Stitched Image")
    plt.axis('off')
    plt.show()
    print("Stitching completed successfully.")
else:
    print("Error in stitching images. Result image is None.")
```

This section is responsible for visualizing the input images and displaying the stitched result. It iterates through the input images, displaying each with a title using Matplotlib. After calling the 'stitch_images' function to obtain the final stitched image, it is displayed if it is not 'None'. Additionally, the stitched image is saved as "stitched_image.jpg," and a success message is printed. If there are no valid images to process or if an error occurs during the stitching process, appropriate messages are printed to indicate the status.

Results and Analysis:

Upon executing the image stitching script, the following results were obtained:

Input Images:

The input images, denoted as 'img11.jpg,' 'img22.jpg,' and 'img33.jpg,' were successfully processed and optimized for resolution. The visual representation of the input images is displayed below:



Fig.1: Input images (Inputs).

Stitched Image:

The stitching process was completed successfully, producing a cohesive panoramic, high-resolution image. The final stitched image is showcased below:



Fig.2: Stitched image (Output).

Observations:

From the result the stitched image effectively combines the visual information from the input images, creating a seamless panorama. The optimization of image resolution ensures compatibility with the constraints of the Xilinx PYNQ board, showcasing adaptability to hardware requirements.

Challenges Faced and Mitigation:

During the implementation of the image stitching solution, several challenges were encountered, including:

1. Hardware Limitations:

The adaptation of the algorithm for the Xilinx PYNQ board posed challenges due to hardware limitations. If the image files were too large the code fails to execute. Optimizing the code to ensure real-time processing and efficient resource utilization was a crucial aspect.

2. Stitching Failures:

The stitching process, relying on OpenCV's Stitcher module, faced occasional failures. Challenges included handling scenarios where images had limited overlapping features, resulting in unsuccessful stitching. A retry mechanism was introduced to address these issues.

3. Algorithm Selection:

Choosing an appropriate image stitching algorithm that aligns with the requirements of real-time AI and the Xilinx PYNQ board presented a decision-making challenge. Balancing performance and accuracy was essential in this context.

Conclusion:

Despite the encountered challenges, the implemented image stitching solution demonstrates successful adaptation to the Xilinx PYNQ board and provides satisfactory results. The iterative process of optimization and troubleshooting has contributed to a more robust and reliable solution for real-time AI applications.